Intrusion Detection System (IDS)

Team IDS: Blake Platt, Mitchell Milander, & Maxwell Reese

WASHINGTON STATE UNIVERSITY

Goal / Project Summary

- The project goal is a network-based Intrusion Detection System (IDS). The system captures live network traffic and processes packets, having data sent from packet capture to traffic analysis, threat detection, and lastly alert generation.
- Currently, the backend is implemented in Python using Scapy for packet sniffing and analysis. Captured packets are processed to extract relevant information such as IP address, ports, packet size, and Protocol flags. A detection engine evaluates this data to identify suspicious behavior.

Client Needs/Requirements

Functional

- Capture live packets using Npcap
- Display packets in real time in a GUI
- Extract packet features (IP, port, protocol, flags)
- Detect suspicious traffic using rule-based detection
- Generate and display alerts in the UI
- Allow user interaction (select packet, inspect details)
- Use test packets to validate detection logic

Non-Functional

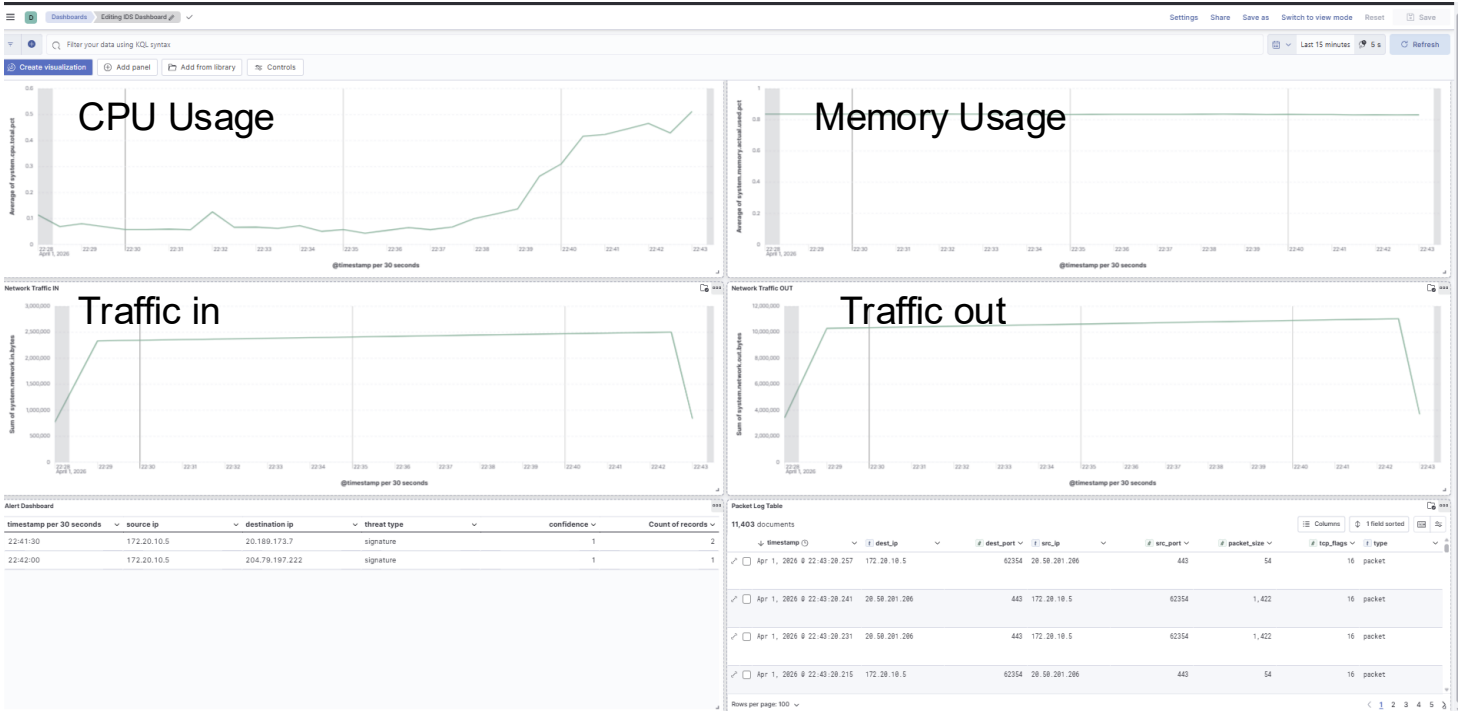
- System must process packets in near real-time
- UI must remain responsive during packet capture
- System should minimize packet loss
- IDS should maintain a low false-positive rate
- Application should run on standard consumer hardware
- Code should be maintainable and modular

Components

- User Interface
- Packet Capture System
- Traffic Analyzer
- Detection Engine

Component 1: User Interface (Elasticsearch)

- Displays packet list in table format
- Allows packet inspection
- Shows alerts in real time
- Provides start/stop capture controls



Component 2: Packet Capture

- Captures packets from selected interface
- Uses Npcap backend for Windows compatibility
- Filters TCP/UDP/IP traffic
- Sends packets to processing queue

```python
class PacketCapture:
    def __init__(self):
        self.packet_queue = queue.Queue()
        self.stop_capture = threading.Event()

    def packet_callback(self, packet):
        original = packet

        # Walk down the payload chain until we find IP
        while packet and not packet.haslayer(IP):
            packet = packet.payload

        if packet and packet.haslayer(IP) and packet.haslayer(TCP):
            print("PACKET:", packet.summary())
            self.packet_queue.put(packet)
        else:
            # Debug print so we know what we're seeing
            print("NON-IP PACKET:", original.summary())




    def start_capture(self, interface):
        def capture_thread():
            sniff(iface=interface,
                  prn=self.packet_callback,
                  store=0,
                  stop_filter=lambda _: self.stop_capture.is_set())

        self.capture_thread = threading.Thread(target=capture_thread)
        self.capture_thread.start()

    def stop(self):
        self.stop_capture.set()
        self.capture_thread.join()
```

Component 3: Traffic Analyzer

- Extracts features from packets
- Tracks flow statistics (packet count, rate, duration)

```python
class TrafficAnalyzer:
    def __init__(self):
        self.port_hits = defaultdict(set)
        self.connections = defaultdict(list)
        self.flow_stats = defaultdict(lambda: {
            'packet_count': 0,
            'byte_count': 0,
            'start_time': None,
            'last_time': None
        })

    def analyze_packet(self, packet):
        if IP in packet and TCP in packet:
            ip_src = packet[IP].src
            ip_dst = packet[IP].dst
            port_src = packet[TCP].sport
            port_dst = packet[TCP].dport

            self.port_hits[ip_src].add(port_dst)

            flow_key = (ip_src, ip_dst, port_src, port_dst)

            # Update flow statistics
            stats = self.flow_stats[flow_key]
            stats['packet_count'] += 1
            stats['byte_count'] += len(packet)
            current_time = packet.time

            if not stats['start_time']:
                stats['start_time'] = current_time
            stats['last_time'] = current_time

            return self.extract_features(packet, stats)

    def extract_features(self, packet, stats):
        duration = stats['last_time'] - stats['start_time']
        if duration == 0:
            duration = 1e-6  # tiny non-zero value to avoid division by zero

        return {
            'packet_size': len(packet),
            'flow_duration': duration,
            'packet_rate': stats['packet_count'] / duration,
            'byte_rate': stats['byte_count'] / duration,
            'tcp_flags': packet[TCP].flags,
            'window_size': packet[TCP].window,
            'packet_count' : stats['packet_count'],
            'source_ip' : packet[IP].src
        }
```

Component 4: Detection Engine

- Applies rule-based detection (e.g., SYN flood, port scan)
- Flags suspicious packets

```python
'port_scan': {
    'condition': lambda features: (
        len(self.traffic_analyzer.port_hits[features['source_ip']]) >= 5 and
        features['packet_rate'] > 100 and
        features['flow_duration'] > 0.01
```

```python
'syn_flood': {
    'condition': lambda features: (
        features['tcp_flags'] == 2 and
        features['packet_rate'] > 50 and
        features['flow_duration'] > 0.005 and
        features['packet_count'] > 3
```

```python
class DetectionEngine:
    def __init__(self, traffic_analyzer):
        self.traffic_analyzer = traffic_analyzer
        self.anomaly_detector = IsolationForest(
            contamination=0.1,
            random_state=42
        )
        self.signature_rules = self.load_signature_rules()
        self.training_data = []
```

```python
def train_anomaly_detector(self, normal_traffic_data):
    self.anomaly_detector.fit(normal_traffic_data)

def detect_threats(self, features):
    threats = []

    # Signature-based detection
    for rule_name, rule in self.signature_rules.items():
        if rule['condition'](features):
            threats.append({
                'type': 'signature',
                'rule': rule_name,
                'confidence': 1.0
            })

    # Anomaly-based detection
    feature_vector = np.array([[
        features['packet_size'],
        features['packet_rate'],
        features['byte_rate']
    ]])

    anomaly_score = self.anomaly_detector.score_samples(feature_vector)[0]
    if anomaly_score < -0.5:  # Threshold for anomaly detection
        threats.append({
            'type': 'anomaly',
            'score': anomaly_score,
            'confidence': min(1.0, abs(anomaly_score))
        })

    return threats
```

mvr_xchange.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

mvrxchange

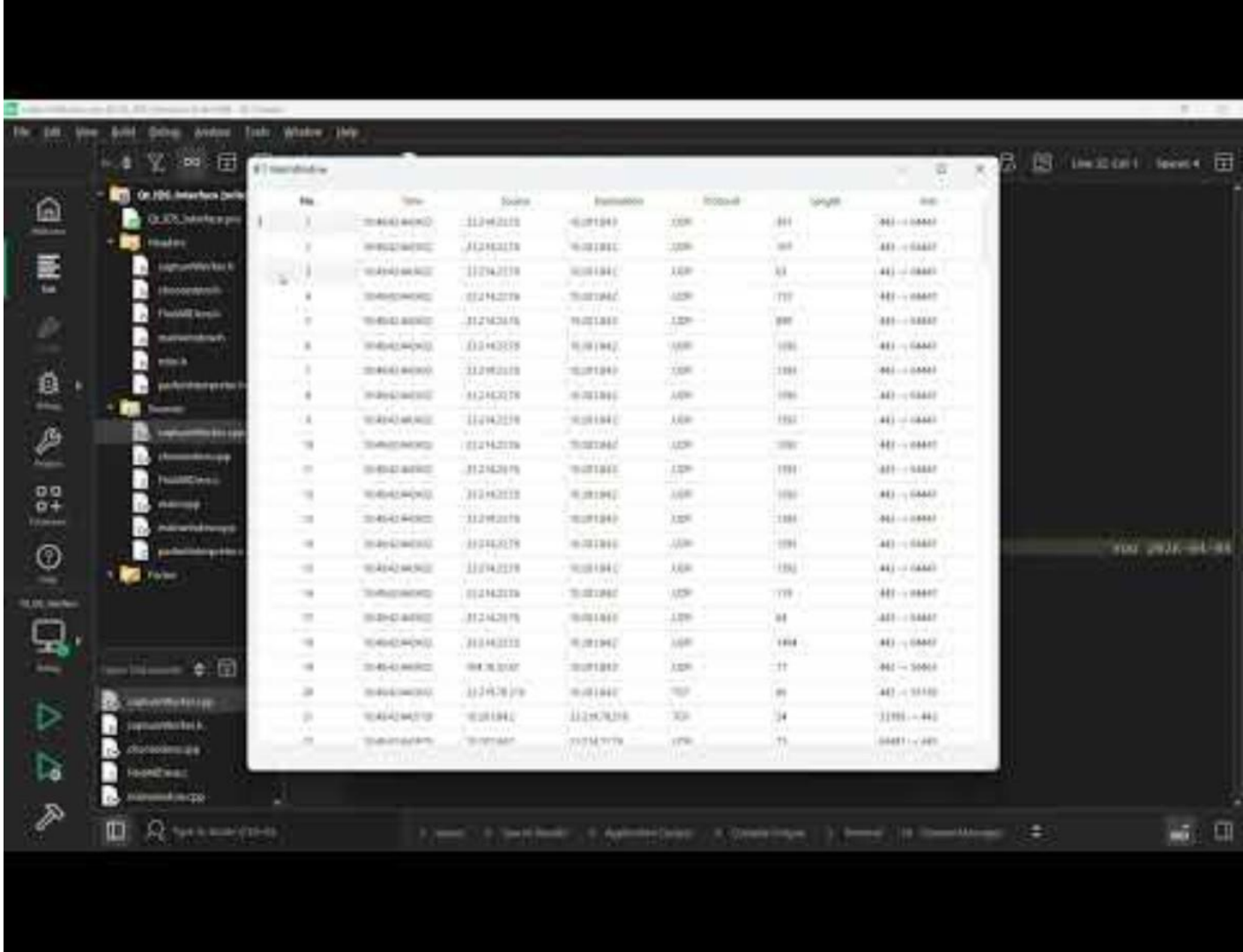
No.	Time	Source	Destination	Protocol	Length	Info
1	17:36:49.1759237…	192.168.20.160	192.168.20.160	TCP	281	34344 → 43997 [PSH, ACK] Seq=1 A…
2	17:36:49.1811649…	192.168.20.160	192.168.20.160	TCP	319	43997 → 34344 [PSH, ACK] Seq=1 A…
3	17:36:54.6431634…	192.168.20.160	192.168.20.160	TCP	303	59222 → 38125 [PSH, ACK] Seq=1 A…
4	17:36:54.6434451…	192.168.20.160	192.168.20.160	TCP	136	38125 → 59222 [PSH, ACK] Seq=1 A…
5	17:36:57.5868388…	192.168.20.160	192.168.20.160	TCP	230	55844 → 43997 [PSH, ACK] Seq=1 A…
6	17:36:57.5925410…	192.168.20.160	192.168.20.160	TCP	16708	43997 → 55844 [ACK] Seq=1 Ack=16…
7	17:37:07.3479695…	192.168.20.160	192.168.20.160	TCP	195	33402 → 43997 [PSH, ACK] Seq=1 A…
8	17:37:07.3492278…	192.168.20.160	192.168.20.160	TCP	98	43997 → 33402 [PSH, ACK] Seq=1 A…

```
▸ Frame 5: 230 bytes on wire (1840 bits), 230 bytes captured (1840 bits)
▸ Linux cooked capture v1
▸ Internet Protocol Version 4, Src: 192.168.20.160 (192.168.20.160), Dst
▸ Transmission Control Protocol, Src Port: 55844 (55844), Dst Port: 4399
▾ MVRxchange
    HEADER: 0x000be1ba
    VERSION: 1
    NUMBER: 0
    COUNT: 1
    TYPE: 0
    LENGTH: 134
    REAL_LENGTH: 134
  ▾ MVR_MESSAGE: {"Type": "MVR_REQUEST", "FileUUID": "F60A2E8E-B231-11EE
      MESSAGE_TYPE: MVR_REQUEST
      MESSAGE_FILE_UUID: F60A2E8E-B231-11EE-9875-A0595013B622
      MESSAGE_FROM_STATION_UUID: 8cdc9843-1072-4c99-a54d-318bdf748825
```

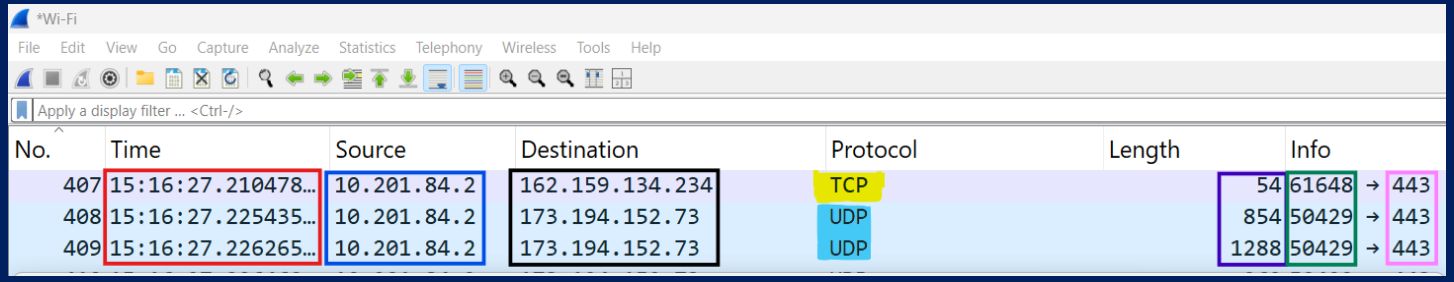
```
0000  00 00 03 04 00 06 00 00  00 00 00 00 02 56 08 00   ........ .....V..
0010  45 00 00 d6 ee c9 40 00  40 06 a0 c7 c0 a8 14 a0   E.....@. @.......
0020  c0 a8 14 a0 da 24 ab dd  e3 6a d5 d3 3f 9d 65 62   .....$.. .j..?.eb
0030  80 18 01 04 ab 59 00 00  01 01 08 0a 18 c5 63 bd   .....Y.. ......c.
0040  18 c5 62 f3 00 0b e1 ba  00 00 00 01 00 00 00 00   ..b..... ........
0050  00 00 00 01 00 00 00 00  00 00 00 00 00 00 00 86   ........ ........
0060  7b 22 54 79 70 65 22 3a  20 22 4d 56 52 5f 52 45   {"Type": "MVR_RE
0070  51 55 45 53 54 22 2c 20  22 46 69 6c 65 55 55 49   QUEST", "FileUUI
0080  44 22 3a 20 22 46 36 30  41 32 45 38 45 2d 42 32   D": "F60 A2E8E-B2
0090  33 31 2d 31 31 45 45 2d  39 38 37 35 2d 41 30 35   31-11EE- 9875-A05
00a0  39 35 30 31 33 42 36 32  32 22 2c 20 22 46 72 6f   95013B62 2", "Fro
00b0  6d 53 74 61 74 69 6f 6e  55 55 49 44 22 3a 20 22   mStation UUID": "
00c0  38 63 64 63 39 38 34 33  2d 31 30 37 32 2d 34 63   8cdc9843 -1072-4c
00d0  39 39 2d 61 35 34 64 2d  33 31 38 62 64 66 37 34   99-a54d- 318bdf74
00e0  38 38 32 35 22 7d                                  8825"}
```

mvr_xchange.pcapng Packets: 8 · Displayed: 8 (100.0%) Profile: Classic

Brief Demo

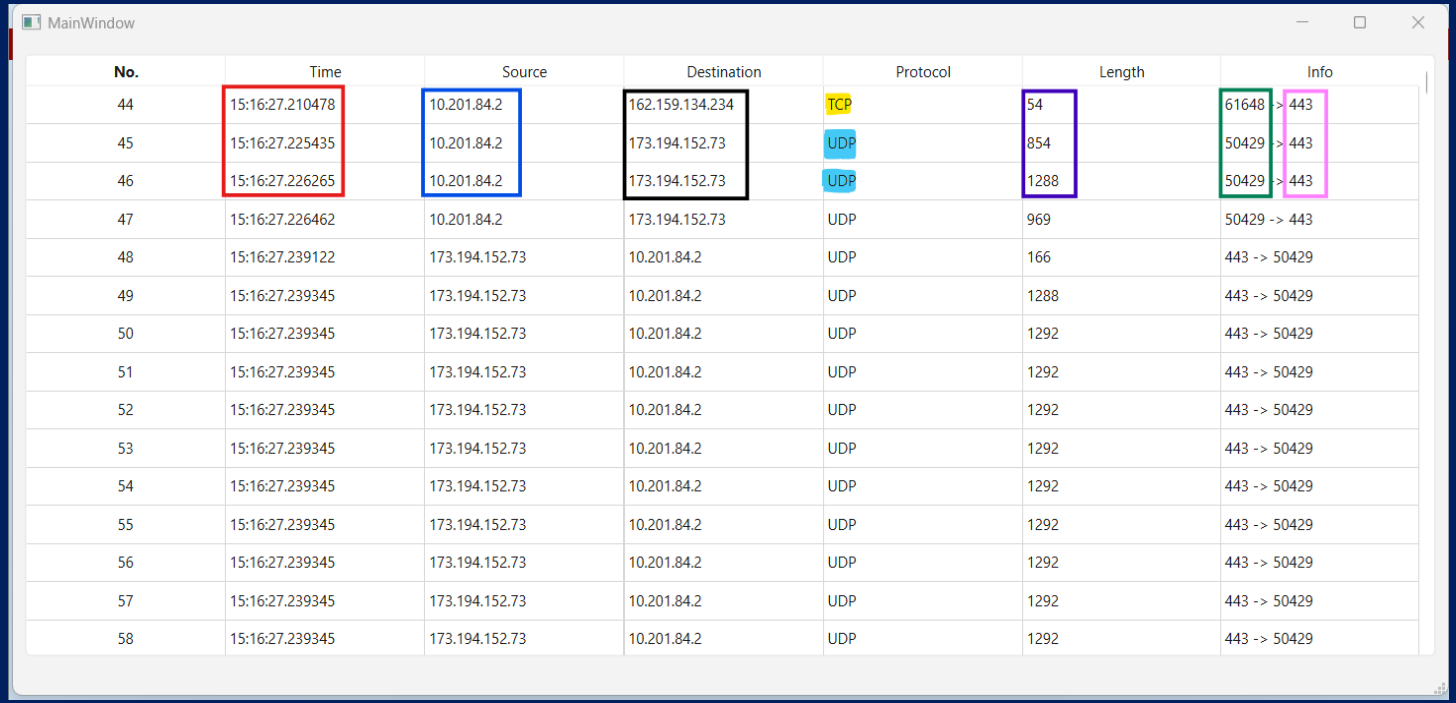


Wireshark



No.	Time	Source	Destination	Protocol	Length	Info
407	15:16:27.210478…	10.201.84.2	162.159.134.234	TCP	54	61648 → 443
408	15:16:27.225435…	10.201.84.2	173.194.152.73	UDP	854	50429 → 443
409	15:16:27.226265…	10.201.84.2	173.194.152.73	UDP	1288	50429 → 443

Qt Interface



No.	Time	Source	Destination	Protocol	Length	Info
44	15:16:27.210478	10.201.84.2	162.159.134.234	TCP	54	61648 -> 443
45	15:16:27.225435	10.201.84.2	173.194.152.73	UDP	854	50429 -> 443
46	15:16:27.226265	10.201.84.2	173.194.152.73	UDP	1288	50429 -> 443
47	15:16:27.226462	10.201.84.2	173.194.152.73	UDP	969	50429 -> 443
48	15:16:27.239122	173.194.152.73	10.201.84.2	UDP	166	443 -> 50429
49	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1288	443 -> 50429
50	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
51	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
52	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
53	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
54	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
55	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
56	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
57	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
58	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429

```
void MainWindow::startInterpreter(QString inter){
    captureThread = new QThread(this);
    CaptureWorker* worker = new CaptureWorker(inter);

    worker->moveToThread(captureThread);
    connect(captureThread, &QThread::started, worker, &CaptureWorker::process);
    connect(worker, &CaptureWorker::packetReady, this, &MainWindow::addPacketRow);
```

```
static void packetBridge(const PacketDetails* pktStruct,void* userData){
    CaptureWorker *worker = static_cast<CaptureWorker*>(userData);
    emit worker->packetReady(QString::fromLocal8Bit(pktStruct->timeStamp),
                             QString::fromLocal8Bit(pktStruct->sourceIP),
```

```
void MainWindow::addPacketRow(QString timeStamp,
                              QString sourceIP,
                              QString destIP,
                              QString proto,
                              int len,
                              int sourcePort,
                              int destPort){
    QScrollBar* bar = ui->home->verticalScrollBar();
    bool wasAtBottom = (bar->value() == bar->maximum());
```

Tools / Frameworks / Platforms / Libraries

Core Development

- **Python** – Backend IDS logic and packet processing
- **Visual Studio Code** – Development environment

Networking & Packet Capture

- **Scapy** – Packet capture and analysis
- **Npcap** – Low-level packet capture support

User Interface

- **Qt** – Desktop interface for real-time monitoring

Data Processing & Detection

- **NumPy** – Anomaly detection
- **Scikit-learn** – Anomaly detection

Data Visualization

- **Elasticsearch** – Data storage/indexing
- **Kibana** – Dashboard UI

Project’s Current Status

- ✅ Packet Sniffer
- ✅ Signature Detection
- ✅ Elastic Dashboard
- ✅ Wireshark-esque Packet Readout

Project’s Future Work

- ❑ Figure out a way for Qt code to work with the Elastic/Python code
- ❑ Implement more Signature-based rules
- ❑ Add Anomaly-based detection functionality
- ❑ Add "Honey Pot" functionality